

```

# =====
# BLOB Principe - SAUT 4 : SENS COMME INSTINCT DU DEPASSEMENT HERITE
# =====
#
# Objectif : tester 1 hypothese philosophique [S] que "le sens est 1 instinct
# du depassement herite" en formulation operationnelle :
#
#   sigma_h(d) = <S_local(descendant) - S_local(ancetre a profondeur d)>
#
# Si sigma_h(d) > 0 et croit avec d -> le Blob a une "pousee" : chaque generation
# est en moyenne plus viable que la generation parent. Cette pousee est ABSENTE
# dans le mode Null (par construction : pas de pression de selection).
#
# IMPLEMENTATION CLE :
#   - Lors de la mitose, on enregistre la genealogie : (descendant -> parent)
#   - A chaque palier, on calcule S_local(v) pour chaque sommet v
#     S_local(v) = log(1 + n_tri_local(v)) - beta * D_local(v)
#     ou n_tri_local(v) = nombre de triangles incidents a v
#     D_local(v) = std/mean des degres des voisins de v
#   - On compare S_local(descendants) a S_local(ancetres) sur plusieurs profondeu
#
# Tests :
#   T1 - sigma_h(1) > 0 : effet immediat parent->enfant
#   T2 - sigma_h(d) croit avec d : effet cumule sur plusieurs generations
#   T3 - sigma_h Blob >> sigma_h Null : signature Blob-specifique
#   T4 - Reproductibilite sur 3 seeds, CV < 30%
#
# Configuration :
#   N final = 1000 (plus rapide), 3 seeds Blob + 3 seeds Null
#   Profondeurs testees : d = 1, 2, 3, 5, 10
# =====

```

```
import sys
```

```
import os
```

```

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None

```

```

    if source_file and os.path.exists(source_file):
        with open(source_file, 'r', encoding='utf-8') as f:
            content = f.read()
            dangerous = {chr(8216): "U+2018", chr(8217): "U+2019",
                          chr(8220): "U+201C", chr(8221): "U+201D",
                          chr(8211): "U+2013", chr(8212): "U+2014"}
            found = [(name, content.count(c)) for c, name in dangerous.items() if
                      if found:
                        print("ERREUR caracteres corrompus :", found)
                        return False
            print("OK : pas de corruption")
        else:
            print("Note : auto-test partiel")
    except Exception as e:
        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

import numpy as np
import time
import json
import gc
from collections import Counter, defaultdict

KAPPA_0 = 8.0
BETA_VIAB = 0.30
TAU = 0.50
P_MITOSE = 0.001
EPS_DECAL = 0.05

OUTPUT_DIR = '/content/blob_saut4_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

class BlobUniverseGrowingWithGenealogy:
    """Blob 2.0 cosmogonique avec tracking genealogique complet."""

    def __init__(self, seed=0, mode='blob'):
        self.rng = np.random.default_rng(seed)
        self.seed = seed
        self.mode = mode
        self.positions = [np.array([0.5, 0.5, 0.5])]
        self.adj = [set()]

```

```

self.deg = np.array([0], dtype=np.int32)
self.edge_list = []
self.n_tri = 0
self.n_tetra = 0
self.S_curr = 0.0
# GENEALOGIE
self.parent_of = {0: None} # parent_of[i] = j signifie i descendant de j
self.birth_step = {0: 0} # pas auquel le sommet i est ne

@property
def N(self):
    return len(self.positions)

@property
def E(self):
    return len(self.edge_list)

def _compute_D(self):
    if self.N == 0:
        return 0.0
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    return np.log(1 + self.n_tri) - BETA_VIAB * self._compute_D()

def _rips_radius(self):
    if self.N <= 1:
        return 0.5
    c_3 = 4 * np.pi / 3
    return 1.5 * (KAPPA_0 / (c_3 * self.N)) ** (1.0/3)

def _count_tetra_through_edge(self, u, v):
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    cl = list(common)
    count = 0
    for i, w in enumerate(cl):
        adj_w = self.adj[w]
        for x in cl[i+1:]:
            if x in adj_w:
                count += 1
    return count

```

```

def mitose(self, current_step):
    parent_idx = int(self.rng.integers(self.N))
    parent_pos = self.positions[parent_idx]
    r_p = self._rips_radius()
    decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
    new_pos = parent_pos + decal
    new_idx = self.N
    self.positions.append(new_pos)
    self.adj.append(set())
    self.deg = np.append(self.deg, 0)
    # GENEALOGIE
    self.parent_of[new_idx] = parent_idx
    self.birth_step[new_idx] = current_step

def step(self, current_step):
    if self.rng.random() < P_MITOSE:
        self.mitose(current_step)
        return False
    if self.N < 2:
        return False
    if self.E == 0:
        return self._step_add_only()
    E_target = int(KAPPA_0 * self.N / 2)
    if self.E < E_target:
        deficit = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit:
            return self._step_add_only()
    return self._step_swap()

def _step_add_only(self):
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    if u == v or v in self.adj[u]:
        return False
    u, v = min(u, v), max(u, v)
    dt_add = len(self.adj[u] & self.adj[v])
    self.adj[u].add(v); self.adj[v].add(u)
    self.deg[u] += 1; self.deg[v] += 1
    new_n_tri = self.n_tri + dt_add
    dtetra_add = self._count_tetra_through_edge(u, v)
    new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
    delta_S = new_S - self.S_curr
    if self.mode == 'null':
        accepted = self.rng.random() < 0.5
    else:
        log_alpha = self.N * delta_S / TAU
        accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al

```

```

    if accepted:
        self.edge_list.append((u, v))
        self.n_tri = new_n_tri
        self.n_tetra += dtetra_add
        self.S_curr = new_S
        return True
    else:
        self.adj[u].discard(v); self.adj[v].discard(u)
        self.deg[u] -= 1; self.deg[v] -= 1
        return False

def _step_swap(self):
    idx = int(self.rng.integers(len(self.edge_list)))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)
    self.adj[u_rm].discard(v_rm); self.adj[v_rm].discard(u_rm)
    self.deg[u_rm] -= 1; self.deg[v_rm] -= 1
    dt_add = len(self.adj[u_add] & self.adj[v_add])
    self.adj[u_add].add(v_add); self.adj[v_add].add(u_add)
    self.deg[u_add] += 1; self.deg[v_add] += 1
    dtetra_add = self._count_tetra_through_edge(u_add, v_add)
    new_n_tri = self.n_tri - dt_rm + dt_add
    if new_n_tri < 0:
        self.adj[u_add].discard(v_add); self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1; self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm); self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1; self.deg[v_rm] += 1
        return False
    new_S = np.log(1 + new_n_tri) - BETA_VIAB * self._compute_D()
    delta_S = new_S - self.S_curr
    if self.mode == 'null':
        accepted = self.rng.random() < 0.5
    else:
        log_alpha = self.N * delta_S / TAU
        accepted = (log_alpha >= 0) or (self.rng.random() < np.exp(max(log_al
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)

```

```

        self.S_curr = new_S
        return True
    else:
        self.adj[u_add].discard(v_add); self.adj[v_add].discard(u_add)
        self.deg[u_add] -= 1; self.deg[v_add] -= 1
        self.adj[u_rm].add(v_rm); self.adj[v_rm].add(u_rm)
        self.deg[u_rm] += 1; self.deg[v_rm] += 1
        return False

def compute_S_local(self, v):
    """S_local(v) = log(1 + n_tri_local(v)) - beta * D_local(v).
    n_tri_local(v) = nombre de triangles incidents a v
    D_local(v) = std/mean des degres des voisins de v"""
    if v >= self.N:
        return 0.0
    neighbors = self.adj[v]
    if not neighbors:
        return 0.0
    # n_tri_local : nombre d aretes entre voisins de v
    n_tri_local = 0
    nlist = list(neighbors)
    for i, u in enumerate(nlist):
        adj_u = self.adj[u]
        for w in nlist[i+1:]:
            if w in adj_u:
                n_tri_local += 1
    # D_local : std/mean des degres des voisins
    deg_neighbors = [self.deg[u] for u in neighbors]
    mu = np.mean(deg_neighbors)
    if mu == 0:
        D_local = 0.0
    else:
        D_local = np.std(deg_neighbors) / mu
    return np.log(1 + n_tri_local) - BETA_VIAB * D_local

def get_ancestor_at_depth(self, v, d):
    """Retourne l ancetre a profondeur d. d=0 -> v lui-meme."""
    current = v
    for _ in range(d):
        parent = self.parent_of.get(current)
        if parent is None:
            return None
        current = parent
    return current

def measure_sigma_h_at_depths(sim, depths):

```

```

"""Pour chaque profondeur d dans depths, calcule:
    sigma_h(d) = <S_local(v) - S_local(ancestor(v, d))>
    sur tous les sommets v ayant un ancetre a profondeur d.
"""
results = {}
for d in depths:
    diffs = []
    for v in range(sim.N):
        ancestor = sim.get_ancestor_at_depth(v, d)
        if ancestor is None:
            continue
        S_v = sim.compute_S_local(v)
        S_a = sim.compute_S_local(ancestor)
        diffs.append(S_v - S_a)
    if diffs:
        results[d] = {
            'mean': float(np.mean(diffs)),
            'std': float(np.std(diffs)),
            'median': float(np.median(diffs)),
            'n_pairs': len(diffs),
            'frac_positive': float(np.mean(np.array(diffs) > 0)),
        }
    else:
        results[d] = None
return results

def run_one(seed, mode, N_target, depths, max_steps=5_000_000):
    t_start = time.time()
    sim = BlobUniverseGrowingWithGenealogy(seed=seed, mode=mode)

    print(f'    [{mode}-s{seed}] start...', flush=True)

    for step in range(max_steps):
        sim.step(step + 1)
        if sim.N >= N_target:
            break

    print(f'    [{mode}-s{seed}] N={sim.N} atteint en {step+1} pas, calcul sigma_

sigma_h_results = measure_sigma_h_at_depths(sim, depths)

    print(f'    [{mode}-s{seed}] termine en {time.time()-t_start:.0f}s', flush=Tr

    return {
        'seed': seed, 'mode': mode,
        'N_final': sim.N,

```

```

        'final_step': step + 1,
        'n_tri': sim.n_tri,
        'n_tetra': sim.n_tetra,
        'sigma_h': sigma_h_results,
        'time_s': time.time() - t_start,
    }

# =====
# MAIN
# =====
print('')
print('=' * 75)
print('  BLOB - SAUT 4 - SENS = INSTINCT DU DEPASSEMENT HERITE')
print('=' * 75)

N_TARGET = int(os.environ.get('BLOB_N_TARGET', 1000))
N_SEEDS = int(os.environ.get('N_SEEDS', 3))
RUN_NULL = os.environ.get('RUN_NULL', '1') == '1'
MAX_STEPS = int(os.environ.get('MAX_STEPS', 5_000_000))

DEPTHS = [int(x) for x in os.environ.get('DEPTHS', '1,2,3,5,10').split(',')]

print(f'  N_target : {N_TARGET}')
print(f'  Profondeurs testees : {DEPTHS}')
print(f'  Seeds : {N_SEEDS}')
print(f'  Run NULL : {RUN_NULL}')
n_modes = 2 if RUN_NULL else 1
print(f'  Estimation : ~{N_SEEDS * n_modes * (N_TARGET / 1000) * 5:.0f} min')
print('=' * 75)

t_global = time.time()
results_blob = []
results_null = []

print('')
print('  === MODE BLOB ===')
for seed in range(N_SEEDS):
    print(f'')
    print(f'  --- Seed {seed} BLOB ---')
    r = run_one(seed, 'blob', N_TARGET, DEPTHS, MAX_STEPS)
    results_blob.append(r)

# Sauvegarde apres chaque seed
inter = OUTPUT_DIR + '/saut4_RAPPORT_partiel.json'
with open(inter, 'w') as f:
    json.dump({'blob': results_blob, 'null': results_null}, f, default=str)

```

```

gc.collect()

if RUN_NULL:
    print('')
    print('   === MODE NULL ===')
    for seed in range(N_SEEDS):
        print(f'')
        print(f'   --- Seed {seed} NULL ---')
        r = run_one(seed, 'null', N_TARGET, DEPTHS, MAX_STEPS)
        results_null.append(r)
        inter = OUTPUT_DIR + '/saut4_RAPPORT_partiel.json'
        with open(inter, 'w') as f:
            json.dump({'blob': results_blob, 'null': results_null}, f, default=st
gc.collect()

total_time = time.time() - t_global
print('')
print('=' * 75)
print(f' Toutes mesures terminees en {total_time/60:.1f} min')
print('=' * 75)

# =====
# AGREGATION ET TESTS
# =====
print('')
print('   --- sigma_h Blob par profondeur ---')
print(f'   {"d":>3s}   {"sigma_h_mean":>14s}   {"std":>8s}   {"frac>0":>8s}   {"n_pair
print('   ' + '-' * 50)

sigma_h_blob_aggreg = {d: [] for d in DEPTHS}
for r in results_blob:
    for d, sh in r['sigma_h'].items():
        if sh:
            sigma_h_blob_aggreg[int(d)].append(sh['mean'])

print(' Moyennes sur seeds :')
for d in DEPTHS:
    vals = sigma_h_blob_aggreg[d]
    if vals:
        m = np.mean(vals)
        s = np.std(vals)
        cv = s/abs(m)*100 if m != 0 else 0
        # frac>0 moyen
        frac_pos_avg = np.mean([r['sigma_h'][d]['frac_positive'] if isinstance(d,
print(f'   d={d:>3d}: sigma_h = {m:.6f} +/- {s:.6f} (CV {cv:.1f}%, frac>0

# Pareil pour Null

```

```

if RUN_NULL:
    sigma_h_null_aggreg = {d: [] for d in DEPTHS}
    for r in results_null:
        for d, sh in r['sigma_h'].items():
            if sh:
                sigma_h_null_aggreg[int(d)].append(sh['mean'])

    print('')
    print(' --- sigma_h Null par profondeur ---')
    for d in DEPTHS:
        vals = sigma_h_null_aggreg[d]
        if vals:
            m = np.mean(vals)
            s = np.std(vals)
            print(f' d={d:>3d}: sigma_h_null = {m:.6f} +/- {s:.6f}')

# COMPARAISON
print('')
print('=' * 75)
print(' COMPARAISON BLOB vs NULL')
print('=' * 75)
if RUN_NULL:
    print(f' {"d":>3s} {"Blob":>14s} {"Null":>14s} {"diff":>14s} {"sig?":>8s}')
    print(' ' + '-' * 60)
    for d in DEPTHS:
        b = sigma_h_blob_aggreg.get(d, [])
        n = sigma_h_null_aggreg.get(d, [])
        if b and n:
            b_mean = np.mean(b)
            n_mean = np.mean(n)
            diff = b_mean - n_mean
            # Test signif simple : diff > 2 * (std_b + std_n)
            std_combine = np.std(b) + np.std(n)
            sig = 'OUI' if abs(diff) > 2 * std_combine else 'limite'
            print(f' d={d:>3d}: {b_mean:>14.6f} {n_mean:>14.6f} {diff:>+14.6f}')

# TESTS T1, T2, T3, T4
print('')
print('=' * 75)
print(' TESTS DE VALIDATION')
print('=' * 75)

# T1 - sigma_h(1) > 0
print('')
sigma_h_1 = sigma_h_blob_aggreg.get(1, [])
if sigma_h_1:
    m = np.mean(sigma_h_1)

```

```

    if m > 0:
        print(f' T1 - sigma_h(1) > 0 : PASS (sigma_h(1) = {m:.6f})')
    else:
        print(f' T1 - sigma_h(1) > 0 : FAIL (sigma_h(1) = {m:.6f})')

# T2 - sigma_h(d) croit avec d
print('')
ds = sorted([d for d in DEPTHS if sigma_h_blob_aggreg.get(d)])
sh_means = [np.mean(sigma_h_blob_aggreg[d]) for d in ds]
if len(sh_means) >= 3:
    diffs_sh = np.diff(sh_means)
    n_inc = sum(1 for x in diffs_sh if x > 0)
    print(f' T2 - sigma_h(d) croit avec d : {n_inc}/{len(diffs_sh)} pas croissan
    if n_inc >= len(diffs_sh) * 0.6:
        print(' PASS (tendance croissante claire)')
    else:
        print(' FAIL (pas de tendance claire)')

# T3 - sigma_h Blob >> Null
print('')
if RUN_NULL:
    ratios = []
    for d in DEPTHS:
        b = sigma_h_blob_aggreg.get(d, [])
        n = sigma_h_null_aggreg.get(d, [])
        if b and n:
            n_abs = abs(np.mean(n))
            if n_abs > 0:
                r = abs(np.mean(b)) / n_abs
                ratios.append(r)
    if ratios:
        ratio_mean = np.mean(ratios)
        print(f' T3 - sigma_h Blob/Null : ratio moyen = {ratio_mean:.2f}')
        if ratio_mean > 2:
            print(' PASS (signal Blob fortement spécifique)')
        elif ratio_mean > 1.2:
            print(' partiel (signal faible)')
        else:
            print(' FAIL (Blob ~ Null)')

# T4 - reproductibilite
print('')
print(f' T4 - Reproductibilite (CV<30% sur 3 seeds par profondeur)')
for d in DEPTHS:
    vals = sigma_h_blob_aggreg.get(d, [])
    if vals:
        m = np.mean(vals)

```

```

        cv = np.std(vals)/abs(m)*100 if m != 0 else 0
        result = 'PASS' if cv < 30 else 'FAIL'
        print(f'      d={d:>3d}: CV = {cv:.1f}% -> {result}')

# VERDICT
print('')
print('=' * 75)
print('  VERDICT GLOBAL')
print('=' * 75)
T1_pass = bool(sigma_h_1 and np.mean(sigma_h_1) > 0)
T3_pass = ratios and np.mean(ratios) > 2 if RUN_NULL and ratios else False
if T1_pass and T3_pass:
    print('  >>> [M] Sens (sigma_h) = signal Blob-spezifique reproductible')
elif T1_pass:
    print('  >>> [H partielle] sigma_h(1) > 0 mais signal pas distingue de Null')
else:
    print('  >>> [FAIL] Hypothese du sens comme depassement herite refutee dans c

# SAUVEGARDE
report = {
    'config': {'N_target': N_TARGET, 'depths': DEPTHS, 'n_seeds': N_SEEDS,
               'run_null': RUN_NULL, 'total_time_s': total_time},
    'results_blob': results_blob,
    'results_null': results_null if RUN_NULL else [],
    'sigma_h_blob_aggregated': {str(d): list(v) for d, v in sigma_h_blob_aggreg.i
    'sigma_h_null_aggregated': {str(d): list(v) for d, v in sigma_h_null_aggreg.i
}
report_path = OUTPUT_DIR + '/saut4_RAPPORT.json'
with open(report_path, 'w') as f:
    json.dump(report, f, indent=2, default=str)
print('')
print(f'  Rapport : {report_path}')
print(f'  Taille : {os.path.getsize(report_path)} octets')

```